Szegmensfa és Fenwick-fa

Szerző: Nikházy László
Előadó: Szente Péter

2026. március 23.

Források:
CP-Algorithms
TopCoder

Szegmensfa — motiváció

Adott egy tömb: $a[0 \ldots n-1]$

Feladatok:

- **Lekérdezés:** $\sum_{i=l}^{r} a[i]$
- **Módosítás:** $a[i] = x$

Naiv megoldások:

- sima tömb:
  - módosítás: $O(1)$
  - lekérdezés: $O(n)$
- prefix összeg:
  - lekérdezés: $O(1)$
  - módosítás: $O(n)$

Cél: mindkettő $O(\log n)$

Szegmensfa — alapötlet

Ötlet: intervallumokat tárolunk egy fában

Nem egyes elemekkel dolgozunk, hanem **szegmensekkel**.

Példa:

$$a[0 \ldots 7]$$

Felosztás:

- teljes tömb
- két fél: $a[0 \ldots 3]$, $a[4 \ldots 7]$
- ezek tovább felezve

Minden csúcs:

- egy intervallumot reprezentál
- eltárolja annak összegét

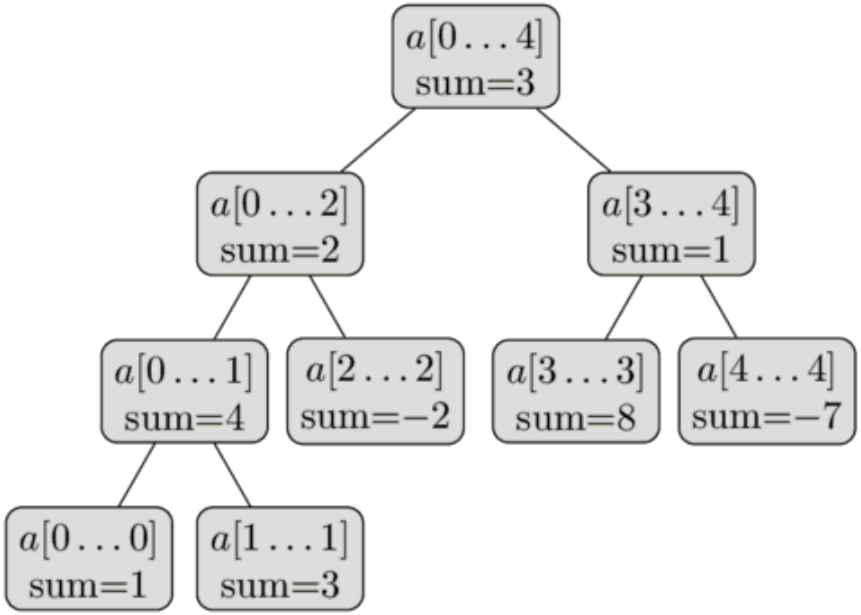
Szegmensfa — struktúra és példa

Struktúra:

- gyökér: `a[0...n-1]`
- minden csúcs egy intervallumot reprezentál
- egy csúcs két gyereke:
  - bal: bal fél
  - jobb: jobb fél
- levelek: egyetlen elem (`a[i]`)

Tárolt érték:

- minden csúcsban az intervallum összege
- belső csúcs = gyerekek összege



Szegmensfa — méret és magasság

Csúcsok száma:

Szintek szerint:

$$1,\ 2,\ 4,\ \ldots,\ 2^k$$

ahol $2^k \geq n$

Összes csúcs:

$$1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1 < 4n$$

Következmény:

- memóriaigény: $O(n)$

Magasság:

- minden lépésben felezzük az intervallumot
- $n \to \frac{n}{2} \to \frac{n}{4} \to \cdots \to 1$

$$\text{magasság} = O(\log n)$$

Megjegyzés:

- ha n nem kettő hatványa, a fa nem teljes
- ez az implementációban lesz fontos

Szegmensfa — reprezentáció

Ötlet: implicit fa tömbben

- csak az értékeket tároljuk
- struktúra indexekből következik

Indexelés (1-alapú):

- gyökér: 1
- bal gyerek: $2v$
- jobb gyerek: $2v + 1$
- szülő: $v/2$

Méret:

- $4n$ elem elég
- lehet, hogy nem minden index használatos
- de egyszerűbb így implementálni

Szegmensfa — reprezentáció

Ötlet: implicit fa tömbben

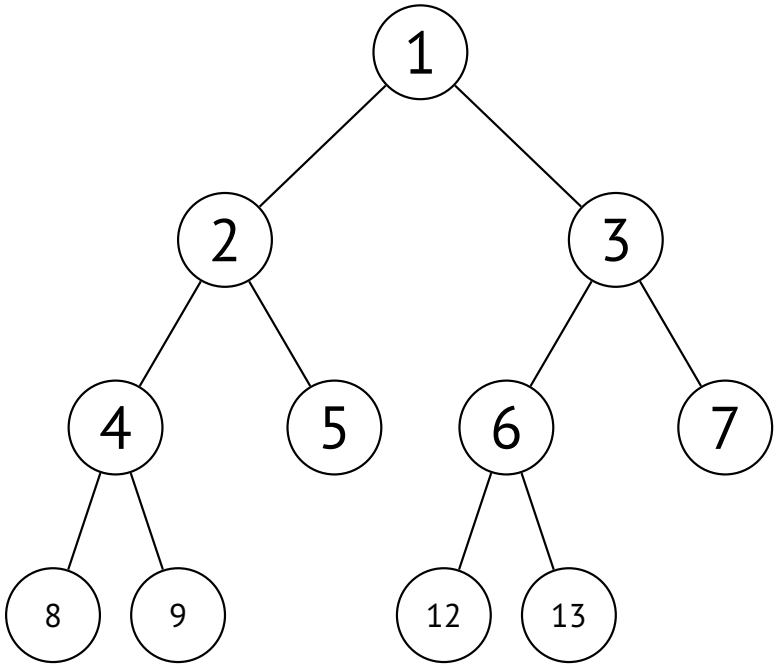
- csak az értékeket tároljuk
- struktúra indexekből következik

Indexelés (1-alapú):

- gyökér: 1
- bal gyerek: $2v$
- jobb gyerek: $2v + 1$
- szülő: $v/2$

Méret:

- $4n$ elem elég
- lehet, hogy nem minden index használatos
- de egyszerűbb így implementálni



Szegmensfa — felépítés

Definiálnunk kell:

- csúcs értéke (pl. összeg)
- kombinálás művelete (pl. összeadás)

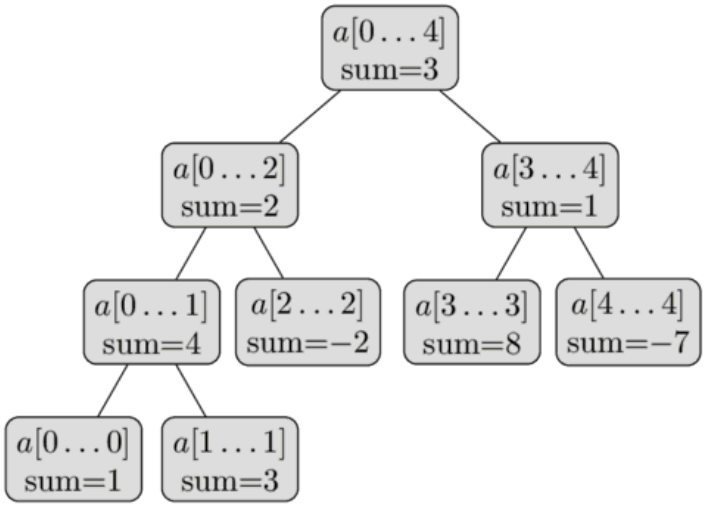
Felépítés:

- levelek: `a[i]`
- belső csúcs: két gyerek kombinálása
- rekurzívan építjük a gyökértől

Rekurzió:

- ha levél: érték = `a[i]`
- különben:
  - bal, jobb részfa
  - kombinálás

Komplexitás: `O(n)`



Szegmensfa — felépítés (kód)

```
1   int n, t[4*MAXN];
2
3   void build(int a[], int v, int tl, int tr) {
4       if (tl == tr) {
5           t[v] = a[tl];
6       } else {
7           int tm = (tl + tr) / 2;
8           build(a, v*2, tl, tm);
9           build(a, v*2+1, tm+1, tr);
10          t[v] = t[v*2] + t[v*2+1];
11      }
12  }
```

Indítás: `build(a, 1, 0, n - 1)`

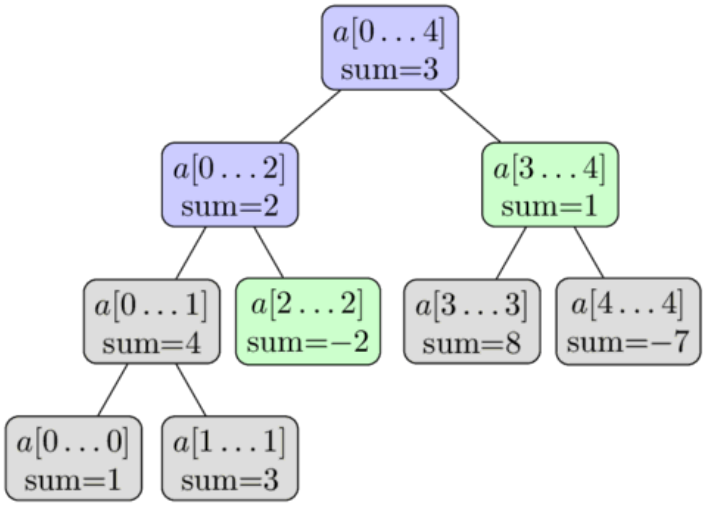
Szegmensfa — lekérdezés

Feladat: $\sum_{i=l}^{r} a[i]$

Aktuális szegmens: `a[tl...tr]`

Esetek:

- **Egyezés:**
  - visszaadjuk az értéket
- **Teljesen bal/jobb:**
  - rekurzió egy irányba
- **Átfedés:**
  - két rekurzió
  - eredmény = összeg



Komplexitás:

- minden szinten legfeljebb 4 csúcsot látogatunk
- $O(\log n)$

Szegmensfa — lekérdezés (kód)

Ötlet:

- mindig két rekurzió
- ha üres: `l > r`

```
1  int sum(int v, int tl, int tr, int l, int r) {
2      if (l > r)
3          return 0;
4      if (l == tl && r == tr) {
5          return t[v];
6      }
7      int tm = (tl + tr) / 2;
8      return sum(v*2, tl, tm, l, min(r, tm))
9           + sum(v*2+1, tm+1, tr, max(l, tm+1), r);
10 }
```

Szegmensfa — frissítés

Feladat: `a[i] = x`

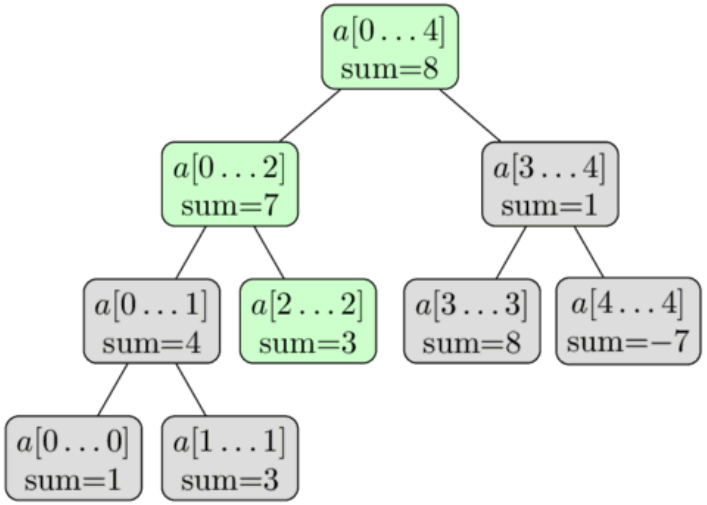
Megfigyelés:

- minden szinten egy szegmens érintett

Algoritmus:

- lemegyünk a levélig
- átírjuk az értéket
- visszafelé:
    - újraszámoljuk (kombinálás)

Komplexitás: `O(log n)`



Szegmensfa — frissítés (kód)

```
1  void update(int v, int tl, int tr,
2              int pos, int new_val) {
3      if (tl == tr) {
4          t[v] = new_val;
5      } else {
6          int tm = (tl + tr) / 2;
7          if (pos <= tm)
8              update(v*2, tl, tm, pos, new_val);
9          else
10             update(v*2+1, tm+1, tr, pos, new_val);
11         t[v] = t[v*2] + t[v*2+1];
12     }
13 }
```

Szegmensfa — általánosítás

Fontos: a szegmensfa nagyon rugalmas

Nem csak összeget tudunk számolni.

Általános séma:

- minden csúcs egy értéket tárol
- van egy **kombinálás** művelet

Példák:

- összeg
- minimum / maximum
- legnagyobb közös osztó
- több adat egy csúcsban

Kulcs:

- a fa struktúrája nem változik
- csak az érték és a kombinálás

Szegmensfa — maximum + darabszám

Feladat:

- maximum és előfordulások száma

Tárolt érték:

- `(max, db)`

Kombinálás:

- nagyobb max $\rightarrow$ azt választjuk
- ha egyenlő:
  - darabszámokat összeadjuk

Továbbra is:

- minden művelet `O(log n)`

Szegmensfa — maximum + darabszám (kód)

```
1  pair<int, int> t[4*MAXN];
2
3  pair<int, int> combine(pair<int, int> a,
4                         pair<int, int> b) {
5      if (a.first > b.first)
6          return a;
7      if (b.first > a.first)
8          return b;
9      return make_pair(a.first,
10                      a.second + b.second);
11 }
12
13 void build(int a[], int v, int tl, int tr) {
14     if (tl == tr) t[v] = {a[tl], 1};
15     else {
16         int tm = (tl + tr) / 2;
17         build(a, v*2, tl, tm);
18         build(a, v*2+1, tm+1, tr);
19         t[v] = combine(t[v*2], t[v*2+1]);
20     }
21 }
```

Szegmensfa — maximum + darabszám (kód)

```
1
2  pair<int,int> get(int v,int tl,int tr,int l,int r){
3      if (l > r) return {-INF, 0};
4      if (l == tl && r == tr) return t[v];
5      int tm = (tl + tr) / 2;
6      return combine(
7          get(v*2, tl, tm, l, min(r, tm)),
8          get(v*2+1, tm+1, tr, max(l, tm+1), r)
9      );
10 }
11
12 void update(int v,int tl,int tr,int pos,int val){
13     if (tl == tr) t[v] = {val, 1};
14     else {
15         int tm = (tl + tr) / 2;
16         if (pos <= tm)
17             update(v*2, tl, tm, pos, val);
18         else
19             update(v*2+1, tm+1, tr, pos, val);
20         t[v] = combine(t[v*2], t[v*2+1]);
21     }
22 }
```

Szegmensfa — nullák számlálása és a k-adik nulla

Feladat:

- Egy tömbben számláljuk a nullák számát adott tartományban.
- Továbbá keressük a `k`-adik nulla indexét.

Módosítás a tárolt értékekben:

- minden csúcs a szegmensben lévő nullák számát tartalmazza

Nullák száma tartományban:

- a count_zero`(l, r)` rekurzió pontosan mint az összeadásnál

K-adik nulla keresése:

- a gyökértől indulunk
- mindig a bal vagy jobb gyerekhez megyünk
- ha a bal gyerek nulláinak száma $\geq$ `k`, megyünk balra
- különben megyünk jobbra, és `k`-ból levonjuk a bal gyerek nulláit

Speciális eset: ha `k` túl nagy, visszaadunk -1

Szegmensfa — k-adik nulla keresése (kód)

```
1  int find_kth_zero(int v, int tl, int tr, int k) {
2      if (k > t[v]) return -1;
3      if (tl == tr) return tl;
4      int tm = (tl + tr) / 2;
5      if (t[v*2] >= k)
6          return find_kth_zero(v*2, tl, tm, k);
7      else
8          return find_kth_zero(v*2+1, tm+1, tr, k - t[v*2]);
9  }
```

Megjegyzés: Ez a rekurzió mindig a megfelelő gyereket választja, így logaritmikus idő alatt megtaláljuk a k-adik nullát.

Szegmensfa — első nagyobb elem keresése

Feladat:

- Adott egy `x` érték és az `a[l...r]` tartomány.
- Keressük a legkisebb `i`-t, ahol `a[i] > x`.

Megoldás:

- Bináris keresés max prefix lekérdezésekkel $O(\log^2 n)$-es megoldást adna
- Hatékonyabb: lefelé haladunk a szegmensfán, balra vagy jobbra lépve a bal gyerek max értékétől függően
- Így $O(\log n)$ idő alatt megtalálható a keresett pozíció

Szegmensfa — első nagyobb elem keresése (kód)

```
1  int get_first_greater(int v, int tl, int tr, int l, int r, int x) {
2      if (tl > r || tr < l) return -1;        // nincs metszet
3      if (t[v] <= x) return -1;               // max nem elég nagy
4
5      if (tl == tr) return tl;                // levél csúcs
6
7      int tm = tl + (tr - tl)/2;
8      int left = get_first_greater(2*v, tl, tm, l, r, x);
9      if (left != -1) return left;
10     return get_first_greater(2*v+1, tm+1, tr, l, r, x);
11 }
```

Megjegyzés: A rekurzió mindig a bal gyereket ellenőrzi először, így garantált a legkisebb index megtalálása.

Szegmensfa — maximális összegű résztartomány

Feladat: Adott tartomány `a[l...r]`, keressük azt az $a[l'...r']$ intervallumot ahol az összege maximális. Az elemek lehetnek negatívak, az optimális szegmens lehet üres.

Megoldás: Minden csúcsban négy értéket tárolunk:

- `sum` — az aktuális szegmens összege
- `pref` — maximális prefix összeg
- `suff` — maximális suffix összeg
- `ans` — maximális résztartomány összege

A kombinálás: egy csúcs értékei a bal és jobb gyerekből számolhatók.

Maximális résztartomány — kombinálás

```
1   struct data {
2       int sum, pref, suff, ans;
3   };
4
5   data combine(data l, data r) {
6       data res;
7       res.sum = l.sum + r.sum;
8       res.pref = max(l.pref, l.sum + r.pref);
9       res.suff = max(r.suff, r.sum + l.suff);
10      res.ans = max(max(l.ans, r.ans), l.suff + r.pref);
11      return res;
12  }
```

Megjegyzés: a kombinálás mindig a négy érték helyes számítására koncentrál.

Maximális résztartomány — felépítés

```
1   data make_data(int val) {
2       data res;
3       res.sum = val;
4       res.pref = res.suff = res.ans = max(0, val);
5       return res;
6   }
7
8   void build(int a[], int v, int tl, int tr) {
9       if(tl == tr) t[v] = make_data(a[tl]);
10      else {
11          int tm = (tl + tr)/2;
12          build(a, v*2, tl, tm);
13          build(a, v*2+1, tm+1, tr);
14          t[v] = combine(t[v*2], t[v*2+1]);
15      }
16  }
```

Maximális résztartomány — lekérdezés és frissítés

```
1  data query(int v, int tl, int tr, int l, int r) {
2      if(l > r) return make_data(0);
3      if(l == tl && r == tr) return t[v];
4      int tm = (tl+tr)/2;
5      return combine(
6          query(v*2, tl, tm, l, min(r, tm)),
7          query(v*2+1, tm+1, tr, max(l, tm+1), r)
8      );
9  }
10
11 void update(int v, int tl, int tr, int pos, int uj) {
12     if(tl == tr) t[v] = make_data(uj);
13     else {
14         int tm = (tl+tr)/2;
15         if(pos <= tm) update(v*2, tl, tm, pos, uj);
16         else update(v*2+1, tm+1, tr, pos, uj);
17         t[v] = combine(t[v*2], t[v*2+1]);
18     }
19 }
```

Teljes szegmensek tárolása a szegmensfában

Ötlet: Minden csúcsban a szegmens összes elemét tároljuk, nem csak összeg/min/max.

- Gyökér: az egész tömb
- Bal gyerek: az első felét tárolja
- Jobb gyerek: a második felét tárolja
- Lehet rendezett lista, `set`, `map`, ...

Memóriaigény: minden elem `O(log n)` csúcsban szerepel, így a teljes fa `O(n log n)` memóriát használ.

Tipikus felhasználások:

- Legkisebb/nagyobb szám egy tartományban
- `k`-adik elem keresése
- Összetettebb lekérdezések rendezett adatokon

Merge Sort Tree — építés

Megközelítés: minden csúcsban az adott szegmens rendezett listája.

```
1   vector<int> t[4*MAXN];
2
3   void build(int a[], int v, int tl, int tr) {
4       if (tl == tr) t[v] = vector<int>(1, a[tl]);
5       else {
6           int tm = (tl+tr)/2;
7           build(a, v*2, tl, tm);
8           build(a, v*2+1, tm+1, tr);
9           merge(t[v*2].begin(), t[v*2].end(),
10                t[v*2+1].begin(), t[v*2+1].end(),
11                back_inserter(t[v]));
12      }
13  }
```

Idő- és memóriaigény: `O(n log n)` mindkettő, mert minden csúcsra a szegmens mérettel arányos.

Lekérdezések Merge Sort Tree-ben

Példa: legkisebb szám, ami nagyobb vagy egyenlő egy adott `x`-nél egy szegmensben.

```
int query(int v, int tl, int tr, int l, int r, int x) {
    if (l > r) return INF;
    if (l == tl && r == tr) {
        auto pos = lower_bound(t[v].begin(), t[v].end(), x);
        return pos != t[v].end() ? *pos : INF;
    }
    int tm = (tl+tr)/2;
    return min(query(v*2, tl, tm, l, min(r, tm), x),
               query(v*2+1, tm+1, tr, max(l, tm+1), r, x));
}
```

Elemzés: minden szegmensnél $O(\log n)$ a bináris keresés, max $O(\log n)$ szegmens $\Rightarrow O(\log^2 n)$ teljes lekérdezés.

Módosítható Merge Sort Tree (multiset)

Probléma a sima Merge Sort Tree-vel: a tömb nem módosítható a lekérdezések között.

Megoldás: minden csúcsban `multiset`, így lehet törölni és beszúrni elemet:

```
1  void update(int v, int tl, int tr, int pos, int new_val) {
2      t[v].erase(t[v].find(a[pos]));
3      t[v].insert(new_val);
4      if (tl != tr) {
5          int tm = (tl+tr)/2;
6          if (pos <= tm) update(v*2, tl, tm, pos, new_val);
7          else update(v*2+1, tm+1, tr, pos, new_val);
8      } else a[pos] = new_val;
9  }
```

Lekérdezés: ugyanaz, $O(\log^2 n)$ **Módosítás:** minden érintett csúcsban frissítjük a `multiset`-et

Szegmensfa — növelés tartományon

Kiindulás:

- Korábban csak egyetlen elemet módosítottunk
- Most: teljes szegmenshez adunk hozzá
  - `a[l...r]+=x`
  - lekérdezés: `a[i]` értéke

Cél: mind frissítés, mind lekérdezés `O(log n)`

Ötlet:

- Nem frissítjük az összes elemet
- Minden csúcsban eltároljuk: mennyit kell hozzáadni az egész szegmenshez

Példa:

- +3 az egész tömbhöz ⇒ csak a gyökérhez írjuk
- Általánosan: `O(log n)` darab szegmensre bontjuk

Lekérdezés:

- Egy `a[i]` értékhez végigmegyünk a gyökértől a levélig
- Összeadjuk az útközben tárolt értékeket

Szegmensfa — növelés tartományon (kód)

```
1   void build(int a[], int v, int tl, int tr) {
2       if (tl == tr) t[v] = a[tl];
3       else {
4           int tm = (tl + tr)/2;
5           build(a, v*2, tl, tm);
6           build(a, v*2+1, tm+1, tr);
7           t[v] = 0;
8       }
9   }
10
11  void update(int v, int tl, int tr, int l, int r, int add) {
12      if (l > r) return;
13      if (l == tl && r == tr) t[v] += add;
14      else {
15          int tm = (tl + tr)/2;
16          update(v*2, tl, tm, l, min(r, tm), add);
17          update(v*2+1, tm+1, tr, max(l, tm+1), r, add);
18      }
19  }
20
21  int get(int v, int tl, int tr, int pos) {
22      if (tl == tr) return t[v];
23      int tm = (tl + tr)/2;
24      if (pos <= tm)
25          return t[v] + get(v*2, tl, tm, pos);
26      else
27          return t[v] + get(v*2+1, tm+1, tr, pos);
28  }
```

Értékadás tartományon (lazy propagation)

Feladat:

- `a[l...r] := p` (minden elem felülírása)
- lekérdezés: `a[i]`

Probléma:

- nem írhatjuk át az összes elemet (`O(n)` lenne)

Ötlet: lustán tároljuk az értékadást

- minden csúcsnál:
  - `t[v]` = az érték
  - `marked[v]` = az egész szegmens egységes
- ha egy csúcs jelölt:
  - az egész szegmens ugyanazt az értéket tartalmazza
  - az alatta lévő részfa „elavult”

Kulcsötlet: csak akkor bontjuk le, ha szükséges

Lazy propagation — push művelet

Probléma:

- ha egy jelölt csúcsból lemegyünk, a gyerekek nem tartalmaznak helyes információt

Megoldás: push

- az értéket „lenyomjuk” a gyerekekhez
- a szülő jelölését töröljük

Mit csinál a push:

- bal gyerek értéke = szülő értéke
- jobb gyerek értéke = szülő értéke
- mindkettőt jelöljük
- szülő jelölés megszűnik

Szabály:

- minden rekurzív lépés előtt push
- csak akkor, ha nem levél

Szegmensfa — lazy propagation (kód)

```
1   void push(int v) {
2       if (marked[v]) {
3           t[v*2] = t[v*2+1] = t[v];
4           marked[v*2] = marked[v*2+1] = true;
5           marked[v] = false;
6       }
7   }
8
9   void update(int v, int tl, int tr, int l, int r, int uj) {
10      if (l > r) return;
11      if (l == tl && tr == r) {
12          t[v] = uj;
13          marked[v] = true;
14      } else {
15          push(v);
16          int tm = (tl+tr)/2;
17          update(v*2, tl, tm, l, min(r, tm), uj);
18          update(v*2+1, tm+1, tr, max(l, tm+1), r, uj);
19      }
20  }
21
22  int get(int v, int tl, int tr, int pos) {
23      if (tl == tr) return t[v];
24      push(v);
25      int tm = (tl+tr)/2;
26      if (pos <= tm)
27          return get(v*2, tl, tm, pos);
28      else
29          return get(v*2+1, tm+1, tr, pos);
30  }
```

Szegmensfa — tartomány növelés + maximum

Feladat:

- növelés: `a[l...r]+=x`
- lekérdezés: `max(a[l...r])`

Mit tárolunk egy csúcsban?

- `t[v]` = maximum a szegmensben

Probléma:

- egy növelés sok elemet érintene → túl lassú lenne

Ötlet:

- nem visszük le azonnal a frissítést a levelekig
- eltároljuk „késleltetve”

Plusz tömb:

- `lazy[v]` = mennyit kell még hozzáadni a gyerekekhez

Lazy propagation — működés

Frissítés egy csúcson:

- `t[v] += x`
- `lazy[v] += x`

Mit jelent a lazy érték?

- a gyerekek még nem kapták meg a növelést

Push (lenyomás):

- mielőtt lemegyünk:
  - bal gyerekhez hozzáadjuk
  - jobb gyerekhez hozzáadjuk
- majd töröljük: `lazy[v] = 0`

Fontos:

- push kell:
  - frissítésnél
  - lekérdezésnél

Tartomány növelés + maximum (kód)

```
1  void push(int v) {
2      t[v*2] += lazy[v];
3      lazy[v*2] += lazy[v];
4      t[v*2+1] += lazy[v];
5      lazy[v*2+1] += lazy[v];
6      lazy[v] = 0;
7  }
8
9  int query(int v, int tl, int tr, int l, int r) {
10     if (l > r) return -INF;
11     if (l == tl && tr == r) return t[v];
12     push(v);
13     int tm = (tl+tr)/2;
14     return max(query(v*2, tl, tm, l, min(r, tm)),
15                query(v*2+1, tm+1, tr, max(l, tm+1), r));
16 }
```

Tartomány növelés + maximum (kód)

```
1  void update(int v, int tl, int tr, int l, int r, int add) {
2      if (l > r) return;
3      if (l == tl && tr == r) {
4          t[v] += add;
5          lazy[v] += add;
6      } else {
7          push(v);
8          int tm = (tl+tr)/2;
9          update(v*2, tl, tm, l, min(r, tm), add);
10         update(v*2+1, tm+1, tr, max(l, tm+1), r, add);
11         t[v] = max(t[v*2], t[v*2+1]);
12     }
13 }
```

Perzisztens szegmensfa

Cél:

- minden módosítás után megőrizzük a régi állapotot
- bármelyik verzión tudunk lekérdezni

Probléma:

- teljes másolás: `O(n)` $\rightarrow$ túl lassú

Kulcsmegfigyelés:

- egy frissítés csak **`O(log n)`** csúcsot érint

Ötlet:

- csak ezeket a csúcsokat másoljuk
- a többi részt újrahasználjuk

Eredmény:

- új verzió: új gyökér
- memória: `O(log n)` / frissítés

Működés — „út másolása”

Adatszerkezet:

- pointeres fa

Frissítéskor:

- gyökértől levélig új csúcsok
- csak az érintett út másolódik

Ami nem változik:

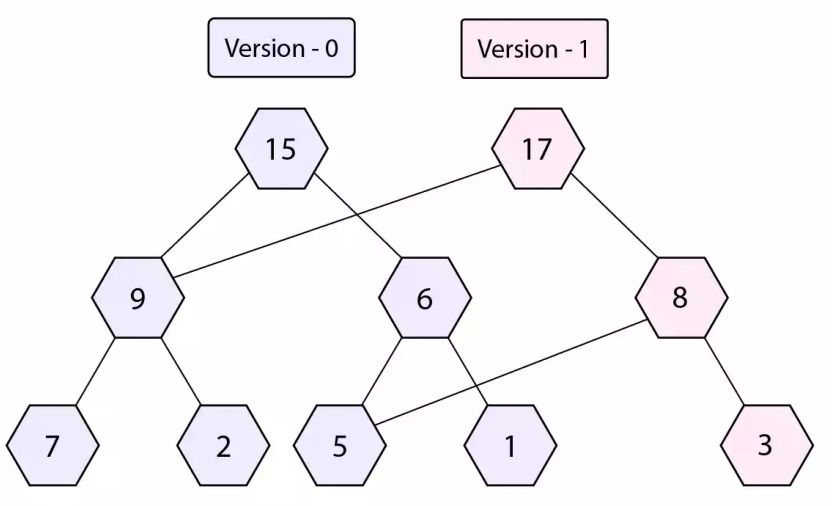
- régi csúcsokra mutatunk

Eredmény:

- régi verzió megmarad
- új verzió részben közös

Verziók kezelése:

- gyökereket tömbben tároljuk



Perzisztens szegmensfa – kód

```
1  struct Vertex {
2      Vertex *l, *r;
3      int sum;
4
5      Vertex(int val) : l(nullptr), r(nullptr), sum(val) {}
6
7      Vertex(Vertex *l, Vertex *r) : l(l), r(r), sum(0) {
8          if (l) sum += l->sum;
9          if (r) sum += r->sum;
10     }
11 };
12
13 Vertex* build(int a[], int tl, int tr) {
14     if (tl == tr)
15         return new Vertex(a[tl]);
16     int tm = (tl + tr) / 2;
17     return new Vertex(build(a, tl, tm),
18                       build(a, tm+1, tr));
19 }
```

Perzisztens szegmensfa — kód

```
1  int query(Vertex* v, int tl, int tr, int l, int r) {
2      if (l > r) return 0;
3      if (l == tl && tr == r)
4          return v->sum;
5      int tm = (tl + tr) / 2;
6      return query(v->l, tl, tm, l, min(r, tm))
7           + query(v->r, tm+1, tr, max(l, tm+1), r);
8  }
9
10 Vertex* update(Vertex* v, int tl, int tr,
11                int pos, int uj) {
12     if (tl == tr)
13         return new Vertex(uj);
14     int tm = (tl + tr) / 2;
15     if (pos <= tm)
16         return new Vertex(
17             update(v->l, tl, tm, pos, uj), v->r);
18     else
19         return new Vertex(
20             v->l, update(v->r, tm+1, tr, pos, uj));
21 }
```

k-adik legkisebb elem intervallumban

Feladat:

- adott $a[l \ldots r]$
- keressük a k-adik legkisebb elemet

Naiv megoldás:

- rendezés $\rightarrow O(n \log n)$
- Merge Sort Tree $\rightarrow O(\log^3 n)$

Cél:

$$O(\log n)$$

Kulcsötlet:

- nem elemeket, hanem **darabszámokat** tárolunk

Ötlet — hisztogram szegmensfa

Átalakítás:

- értéktartomány: $[0 \ldots n-1]$

Mit tárolunk?

- hány darab szám esik egy intervallumba

Levél:

- egy konkrét érték gyakorisága

Belső csúcs:

- összeg $\rightarrow$ hány szám van ebben a tartományban

Így:

- a k-adik elem = hol lesz először legalább k elem addig

Perzisztencia — prefixek

Felépítés:

- $\texttt{root}_i$ = első $\texttt{i}$ elem hisztogramja
- indulás: üres fa ($\texttt{root}_0$)
- sorban beszúrjuk az elemeket

Intervallum lekérdezés:

$$[\texttt{l} \ldots \texttt{r}] = \texttt{root}_\texttt{r} - \texttt{root}_{\texttt{l}-1}$$

Mit jelent ez?

- darabszám = különbség
- minden csúcsban kivonjuk az értékeket

Hogyan találjuk meg a k-adik elemet?

- trükk: két fán egyszerre sétálunk
- nézzük: hány elem van bal oldalon
  - ha $\geq \texttt{k} \rightarrow$ balra megyünk
  - különben: jobbra megyünk és $\texttt{k} := \texttt{k} - \texttt{bal_darab}$

find_kth – kód

```
1  Vertex* build(int tl, int tr) {
2      if (tl == tr)
3          return new Vertex(0);
4      int tm = (tl + tr) / 2;
5      return new Vertex(build(tl, tm),
6                        build(tm+1, tr));
7  }
8
9  Vertex* update(Vertex* v, int tl, int tr, int pos) {
10     if (tl == tr)
11         return new Vertex(v->sum + 1);
12     int tm = (tl + tr) / 2;
13     if (pos <= tm)
14         return new Vertex(
15             update(v->l, tl, tm, pos), v->r);
16     else
17         return new Vertex(
18             v->l, update(v->r, tm+1, tr, pos));
19 }
```

find_kth – kód

```
1  int find_kth(Vertex* vl, Vertex* vr,
2               int tl, int tr, int k) {
3      if (tl == tr)
4          return tl;
5
6      int tm = (tl + tr) / 2;
7      int left = vr->l->sum - vl->l->sum;
8
9      if (left >= k)
10         return find_kth(vl->l, vr->l, tl, tm, k);
11     else
12         return find_kth(vl->r, vr->r, tm+1, tr,
13                         k - left);
14 }
```

Dinamikus szegmensfa

Probléma:

- nagyon nagy tartomány (pl. $[0 \ldots 10^9]$)
- nem tudjuk előre felépíteni a teljes fát

Klasszikus szegmensfa:

- $O(n)$ memória $\rightarrow$ túl nagy

Ötlet:

- csak a szükséges csúcsokat hozzuk létre

Más néven:

- implicit szegmensfa
- ritka (sparse) szegmensfa

Komplexitás:

- műveletek: $O(\log n)$
- memória: csak a használt csúcsok

Működés

Kezdet:

- csak a gyökér létezik

Lépés közben:

- ha kell egy gyerek $\rightarrow$ létrehozzuk

extend():

- létrehozza a két gyereket
- csak ha még nem léteznek

Intuíció:

- a fa „kinő” a lekérdezések mentén

Példa:

- ha csak 3 pozíciót használunk
- csak ezekhez vezető utak léteznek

Dinamikus szegmensfa – kód

```
1   struct Vertex {
2       int left, right;
3       int sum = 0;
4       Vertex *l = nullptr, *r = nullptr;
5
6       Vertex(int lb, int rb) {
7           left = lb;
8           right = rb;
9       }
10
11      void extend() {
12          if (!l && left + 1 < right) {
13              int mid = (left + right) / 2;
14              l = new Vertex(left, mid);
15              r = new Vertex(mid, right);
16          }
17      }
```

Dinamikus szegmensfa – kód

```
1       void add(int pos, int val) {
2           extend();
3           sum += val;
4           if (l) {
5               if (pos < l->right)
6                   l->add(pos, val);
7               else
8                   r->add(pos, val);
9           }
10      }
11
12      int query(int ql, int qr) {
13          if (ql <= left && right <= qr)
14              return sum;
15          if (max(left, ql) >= min(right, qr))
16              return 0;
17          extend();
18          return l->query(ql, qr) + r->query(ql, qr);
19      }
20  };
```

Szegmensfa - Összefoglalás

Lényeg:

- Résztömbökön kell lekérdezéseket végrehajtani, és elemek módosítását kezelni
- Bizonyos résztömbökre eltároljuk az eredményt
- A kérdésben szereplő intervallumot fel tudjuk bontani `O(log n)` előre kiszámolt résztömbre a fa struktúra segítségével
- Két résztömbre kiszámolt eredmény *kombinálását* meg kell tudnunk valósítani
- Egy-egy elem módosításakor `O(log n)` résztömbre kell újraszámolni az eredményt

Változatok:

- Módosítások intervallumokon $\rightarrow$ *lazy propagation*
- Korábbi verziók megörzése $\rightarrow$ *perzisztencia* út másolással
- Nagy tartomány esetén csak a szükséges csúcsok $\rightarrow$ *dinamikus/implicit* szegmensfa

Fenwick-fa

Más néven: Binary Indexed Tree

Fenwick-fa — motiváció

Adott egy `a` tömb.

Műveletek:

- **Frissítés:** `a[i]+=x`
- **Lekérdezés:** $\sum_{i=l}^{r} a[i]$
- **Cél:** mindkettő $O(\log n)$

Fenwick-fa:

- prefix összegeket kezel
- egyszerűbb, mint a szegmensfa

Ötlet:

- számok felbonthatók 2 hatványok összegére
- prefix összeg is felbontható nem átfedő intervallumokra

Jelölések:

- $pref[i] = \sum_{j=1}^{i} a[j]$
- `tree[i]` — egy részösszeg

Fenwick-fa — felelősségi intervallumok

Kulcs:

- minden `i` index egy intervallumért felel

Definíció:

- `lsb(i)` = legkisebb helyiértékű 1 bit

Ekkor:

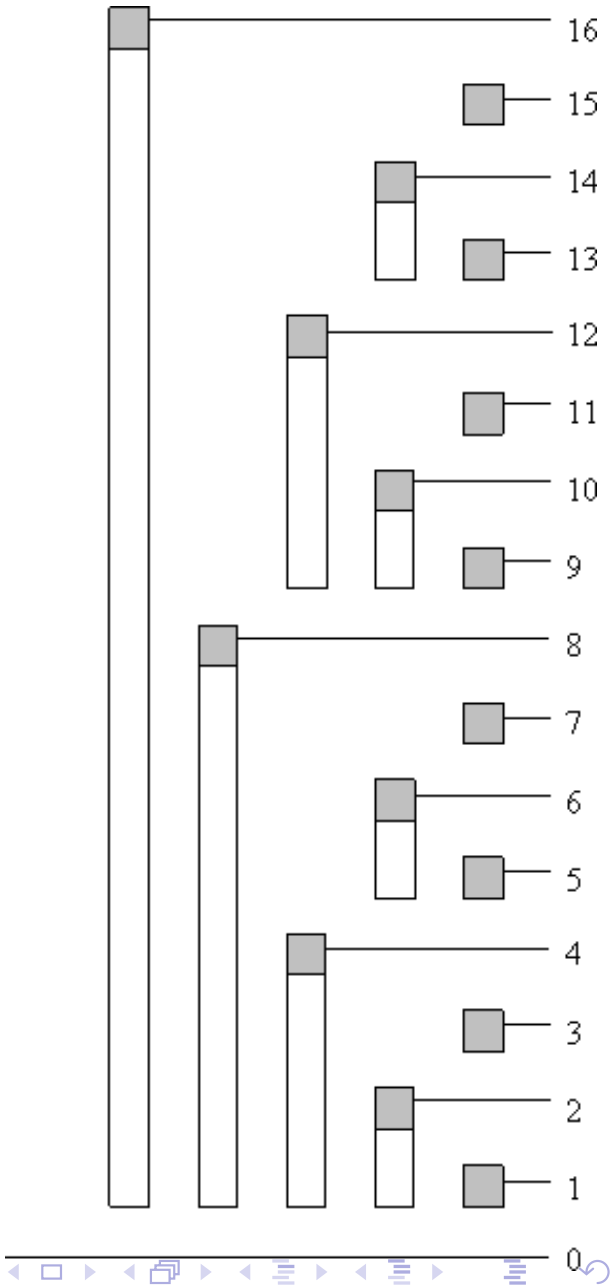
$$\texttt{tree[i]} = \sum_{\texttt{j=i-lsb(i)}+1}^{\texttt{i}} \texttt{a[j]}$$

Jelentés:

- `i` lefed egy intervallumot
- hossza: `lsb(i)`

Cél:

- ezekből építjük fel a prefixet



Fenwick-fa — példa

Feladat: `pref[13]`

Binárisan:

$$13 = 1101$$

Felbontás:

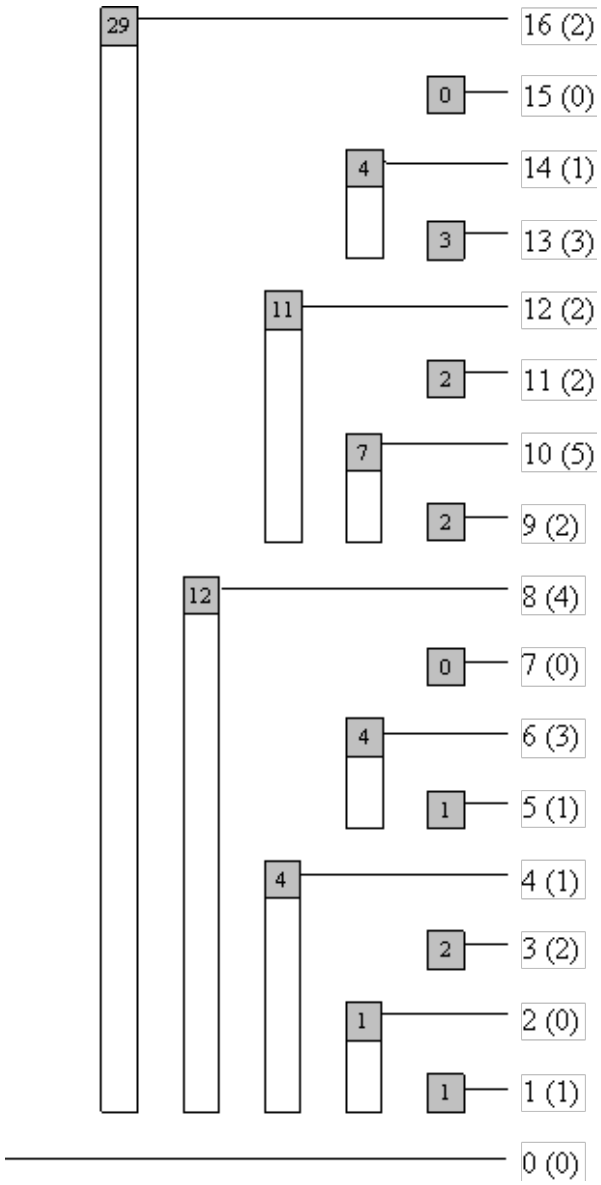
`pref[13] = tree[13]+tree[12]+tree[8]`

Lépés:

- `i`-t mindig `lsb(i)`-vel csökkentjük

Eredmény:

- `O(log n)` darab intervallum



Fenwick-fa — lsb(i) meghatározása

Cél: `lsb(i)` = legkisebb helyiértékű 1 bit

Legyen `i` binárisan:

$$\mathtt{i} = \mathtt{a}1\mathtt{b}$$

ahol `b` csak nullákból áll.

Kettes komplemens:

$$-\mathtt{i} = \overline{\mathtt{i}} + 1 = \overline{\mathtt{a}1\mathtt{b}} + 1 = \overline{\mathtt{a}}0\overline{\mathtt{b}} + 1$$

Mivel $\mathtt{b} = 00\ldots0$:

$$\overline{\mathtt{b}} = 11\ldots1 \Rightarrow -\mathtt{i} = \overline{\mathtt{a}}1\,00\ldots0$$

És:

$$\mathtt{i}\ \&\ (-\mathtt{i}) = \mathtt{a}1\mathtt{b}\ \&\ \overline{\mathtt{a}}1\mathtt{b} = 00\ldots010\ldots0$$

Következmény:

$$\mathtt{lsb}(\mathtt{i}) = \mathtt{i}\ \&\ (-\mathtt{i})$$

Fenwick-fa — prefix lekérdezés

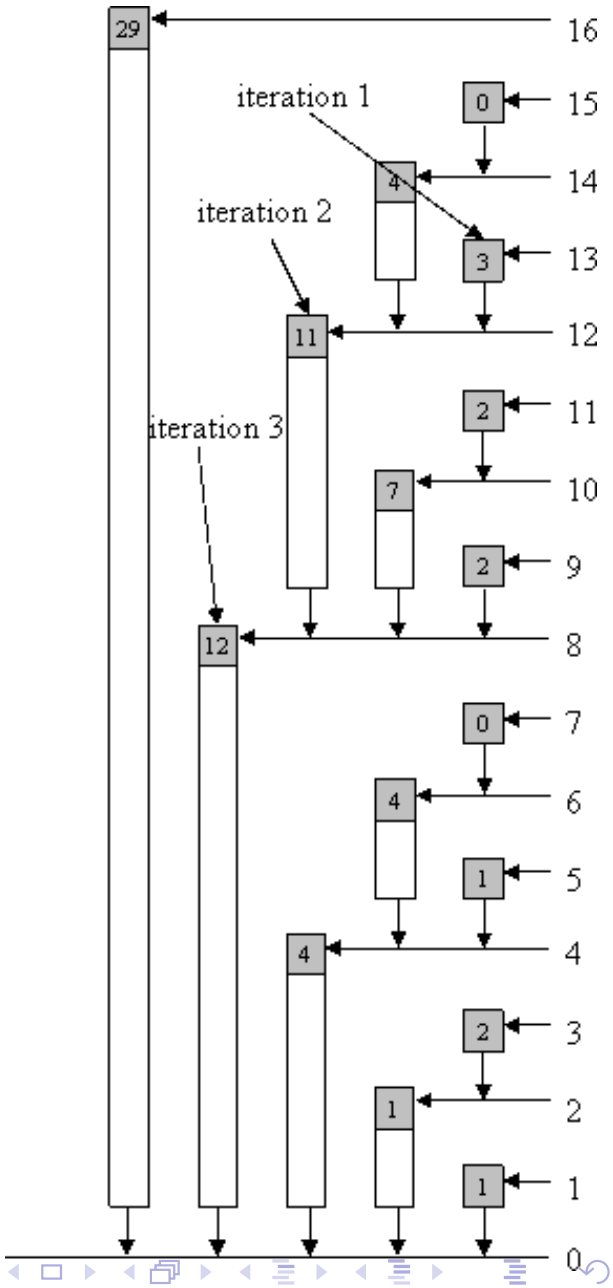
Cél: `pref[i]`

Ötlet:

- `i`-hez tartozó intervallumokat összegezzük
- minden lépésben: `i-=lsb(i)`

Komplexitás: `O(logn)` lépés

```
1  int query(int i) {
2      int sum = 0;
3      while (i > 0) {
4          sum += tree[i];
5          i -= (i & -i);
6      }
7      return sum;
8  }
```



Fenwick-fa — frissítés

Feladat: `a[i]+=x`

Ötlet:

- minden olyan `j`-t frissítünk,
- amelynek intervalluma tartalmazza `i`-t

Algoritmus:

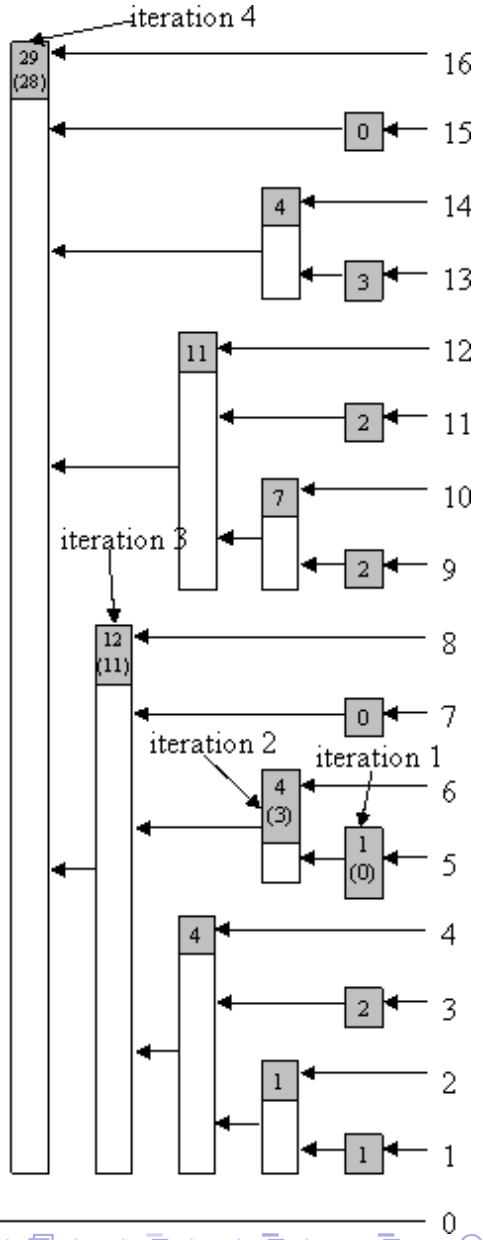
- indulás: `i`
- minden lépésben: `i+=lsb(i)`

Komplexitás: `O(log n)` lépés

Megjegyzés:

- query: `i-=lsb(i)`
- update: `i+=lsb(i)`

```
1  void update(int i, int x) {
2      while (i <= n) {
3          tree[i] += x;
4          i += (i & -i);
5      }
6  }
```



Fenwick-fa — egy elem lekérdezése

Feladat: `a[i]`

Probléma:

- `tree[i]` nem egyetlen elem
- hanem egy intervallum összege

Egyszerű megoldás:

$$\texttt{a[i]} = \texttt{pref[i]} - \texttt{pref[i} - 1\texttt{]}$$

Implementáció:

- két query hívás

Kérdés:

- lehet-e gyorsabban?

Fenwick-fa — egy elem lekérdezése (optimalizált)

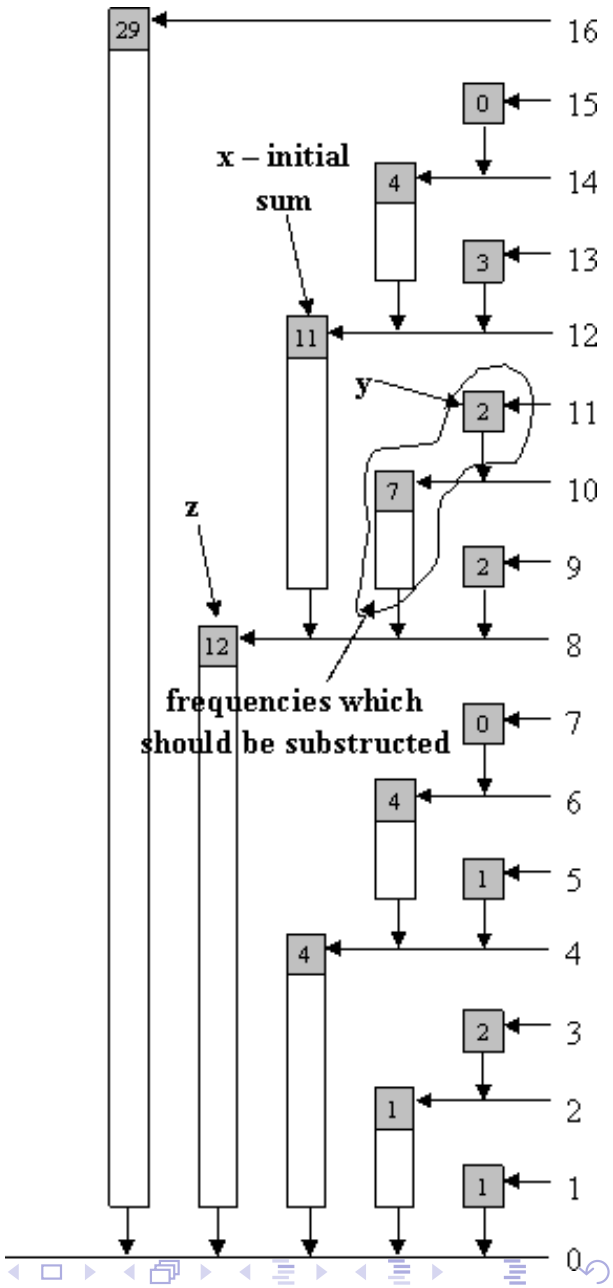
Ötlet:

- két „query útvonal” különbsége
- csak a különböző részeket vonjuk ki

Algoritmus:

- indulás: `sum = tree[i]`
- levonjuk a nem szükséges részeket

```
int get(int i) {
    int sum = tree[i];
    if (i > 0) {
        int z = i - (i & -i);
        i--;
        while (i != z) {
            sum -= tree[i];
            i -= (i & -i);
        }
    }
    return sum;
}
```



Fenwick-fa — összefoglaló

Támogatott műveletek:

- **Frissítés:** `a[i]+=x`
- **Prefix:** `pref[i]`
- **Intervallum:** `[l, r] = pref[r] - pref[l - 1]`

Komplexitás:

- minden művelet: `O(log n)`
- memória: `O(n)`

Mikor használható?

- ha a művelet **prefixekből kifejezhető**
- pl.: összeg, darabszám, XOR

Mikor NEM használható?

- ha nincs „invertálhatóság”, pl.: `min`, `max`
- mert `[l, r]` nem számolható csak prefixekből

Kulcs:

- szükséges egy „kivonás-szerű” művelet
- $F(l, r) = F(r) \ominus F(l-1)$

Feladatok

Codeforces: Subsequences

`https://codeforces.com/contest/597/problem/C`

Adott egy n különböző elemből álló sorozat.

Feladat: Határozzuk meg, hány darab $k + 1$ hosszúságú, szigorúan növekvő részsorozata van a sorozatnak!

Korlátok:

- $1 \leq n \leq 10^5$
- $0 \leq k \leq 10$
- A válasz garantáltan $\leq 8 \cdot 10^{18}$ (elfér `long long`-ban).

Bemenet: Az első sor tartalmazza n és k értékét. A következő n sor mindegyike a sorozat egy-egy elemét (a_i) tartalmazza.

Példa bemenet:

```
5 2
1
2
3
5
4
```

Kimenet:

```
7
```

COCI 2017 Deda

https://oj.uz/problem/view/COCI17_deda

Adott N gyerek, akiket 1-től N-ig számozunk életkoruk szerint (1 a legfiatalabb, N a legidősebb). Egy vonat halad a megállókon keresztül.

Műveletek:

- M X A: Marica állítása szerint az A jelű gyerek az X. megállónál szállt ki a vonatból.
- D Y B: A nagyapa kérdése: a legalább B indexű (korú) gyerekek közül ki a **legfiatalabb**, aki legfeljebb Y megállót utazott?

Korlátok:

- $2 \leq N, Q \leq 200\,000$
- $1 \leq X, Y \leq 10^9$

Bemenet: Az első sor N és Q értékét tartalmazza. A következő Q sor a fenti típusú műveleteket írja le.

Példa bemenet:

```
3 4
M 10 3
M 5 1
D 20 2
D 5 1
```

Kimenet:

```
3
1
```

IOI 2014 Wall

`https://oj.uz/problem/view/IOI14_wall`

Jian-Jia egy falat épít `n` oszlopból, amelyek magassága kezdetben 0. Az építés során `k` fázison keresztül módosítja a falat.

Feladat: Minden fázisban adott egy `[left, right]` tartomány és egy `h` magasság. A művelet típusa szerint:

- **Hozzáadás (add):** A tartomány azon oszlopait, amelyek magassága kisebb, mint `h`, kiegészítjük pontosan `h` magasságra.
- **Eltávolítás (remove):** A tartomány azon oszlopait, amelyek magassága nagyobb, mint `h`, visszabontjuk pontosan `h` magasságra.

Cél: Határozzuk meg az összes oszlop végső magasságát.

Korlátok:

- $1 \leq \texttt{n} \leq 2\,000\,000$
- $1 \leq \texttt{k} \leq 500\,000$
- $0 \leq \texttt{h} \leq 100\,000$

Példa bemenet:

```
10 3
1 1 8 4
2 4 9 1
2 3 6 5
```

Példa kimenet:

```
0 4 4 4 1 1 1 1 1 0
```

IOI 2015 Teams

`https://oj.uz/problem/view/IOI15_teams`

Adott egy $\mathtt{N}$ fős osztály. Minden diákhoz tartozik egy $[\mathtt{A}_i, \mathtt{B}_i]$ intervallum: a diák csak olyan csapatba osztható be, amelynek mérete legalább $\mathtt{A}_i$ és legfeljebb $\mathtt{B}_i$.

Feladat: Dolgozzunk fel $\mathtt{Q}$ napot! Minden nap kapunk $\mathtt{M}$ projektet, ahol minden projekthez megadják a szükséges csapatméretet ($\mathtt{K}_j$). Döntsük el, hogy az adott napon minden projekt kérése teljesíthető-e különböző diákokkal!

Korlátok:

- $1 \leq \mathtt{N} \leq 500\,000$
- $1 \leq \mathtt{Q} \leq 200\,000$
- A csapatméretek összege ($\sum \mathtt{K}_j$) az összes napra nézve $\leq 500\,000$.